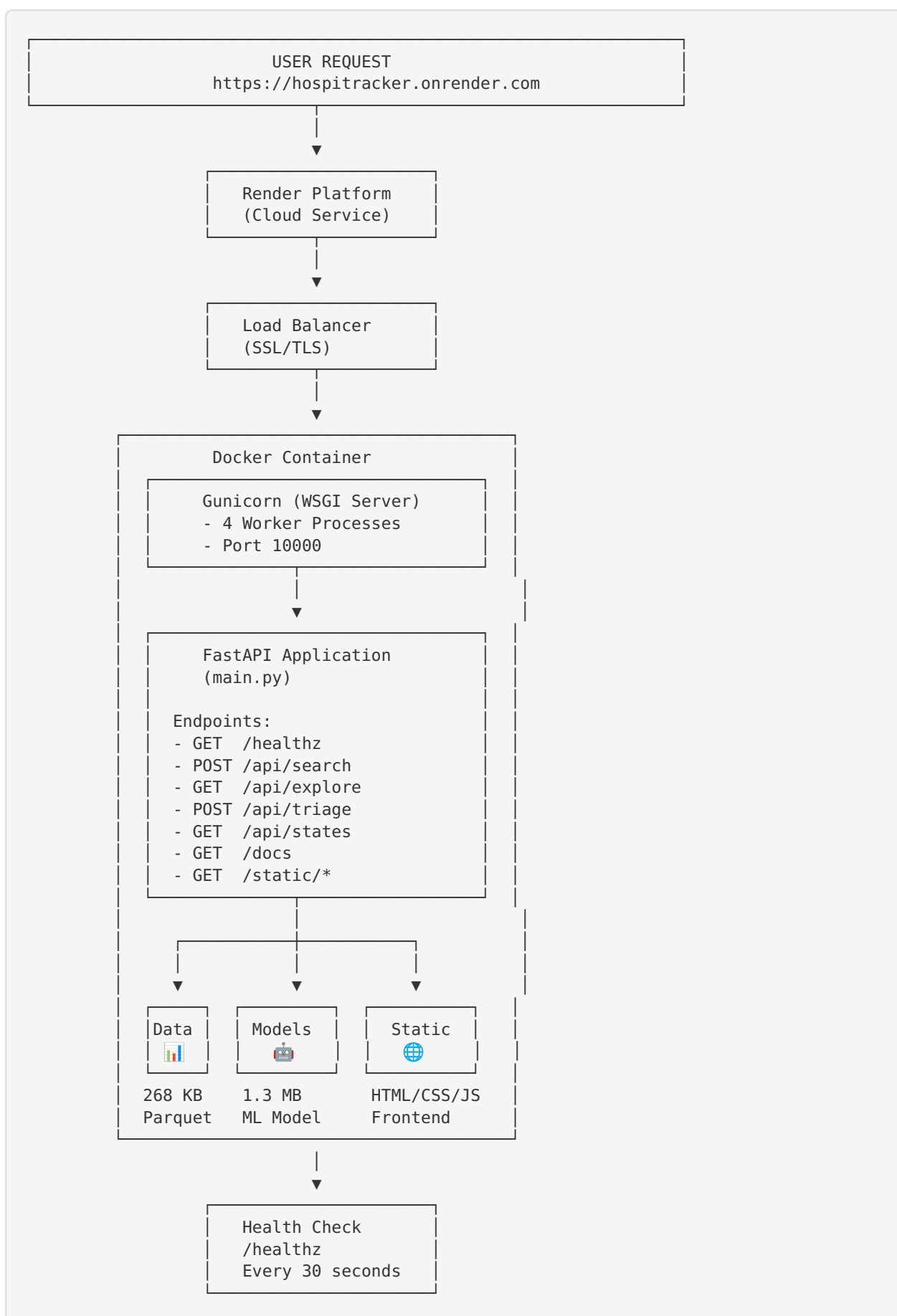


HospiTrack Deployment Flow

Architecture Diagram



Deployment Pipeline

Step 1: LOCAL DEVELOPMENT

/home/ubuntu/hospitracker/

```

main.py           (FastAPI backend)
Dockerfile.prod   (Production image)
render.yaml       (Render config)
data/
├── us_er.parquet  (4,088 hospitals)
├── models/
│   └── triage_model.pkl (ML model)
└── static/
    ├── index.html   (Landing)
    ├── home.html    (Find Care)
    ├── results.html (Map + List)
    ├── explore.html (Browse)
    └── demo.html     (Triage)
  
```

Status: ☒ All bug fixes completed
☒ 10 commits ready to push
☒ Deployment files ready

☐ ☐ git push origin main
☒

Step 2: GITHUB REPOSITORY

<https://github.com/JZembower/HospiTrack>

Actions:

- Store source code
- Version control
- Trigger CI/CD on push

Status: ☒ Waiting **for** git push

☐ ☐ Webhook trigger
☒

Step 3: RENDER BUILD

Build Process (5-10 minutes):

1. Clone repository ☒ 30 seconds
2. Build Docker image ☒ 5 minutes
 - FROM **python:3.11-slim**
 - Install dependencies
 - COPY data/ models/ static/
 - RUN pip install -r requirements_fastapi.txt
3. Push to registry ☒ 2 minutes
4. Deploy container ☒ 1 minute
5. Health check ☒ 30 seconds

Build Logs Available: [Render Dashboard](#) ☒ [Logs](#)

☐ ☐ Deployment complete

▼

Step 4: PRODUCTION (LIVE)

 Public URL: <https://hospitracker-XXXX.onrender.com>

Features:

☒ Landing page with feature overview

☒ Find Care - Search by symptoms **and** location

☒ Results - Interactive map + hospital list

☒ Explore - Browse 4,088 hospitals nationwide

☒ Demo - ML-powered triage simulation

☒ API Docs - Interactive Swagger UI

Performance:

- Health **check**: /healthz every 30s

- Memory **usage**: ~200 MB average

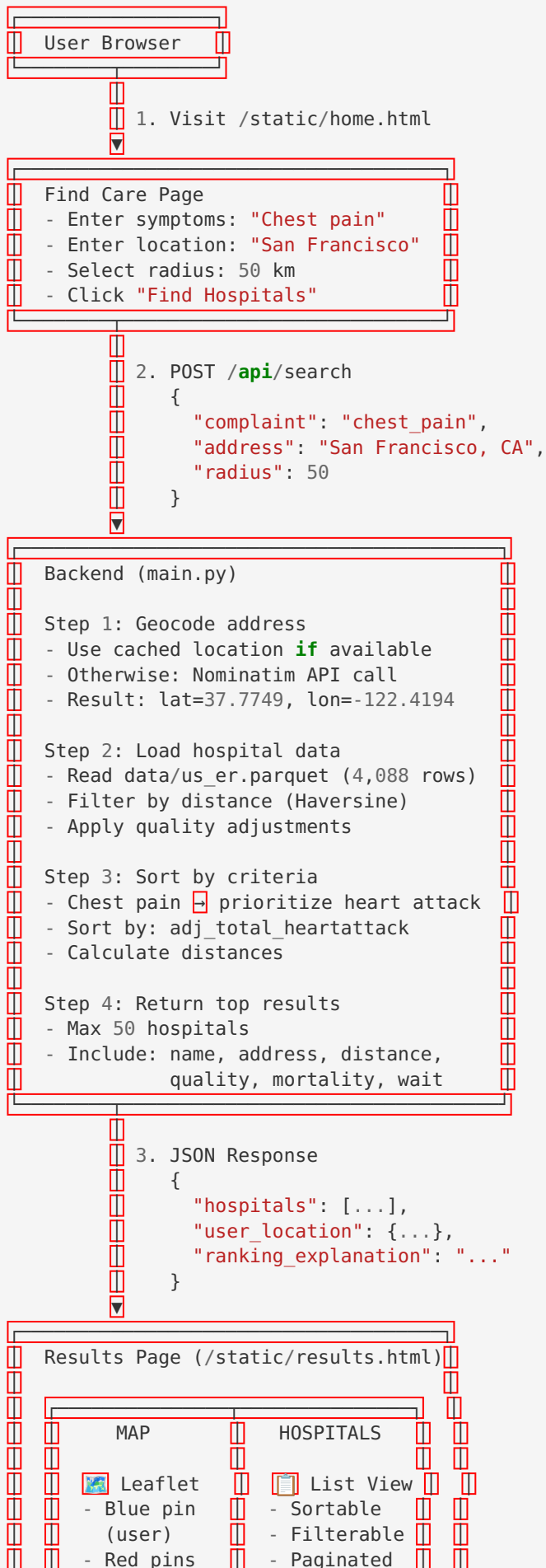
- Response **time**: <2s **for** searches

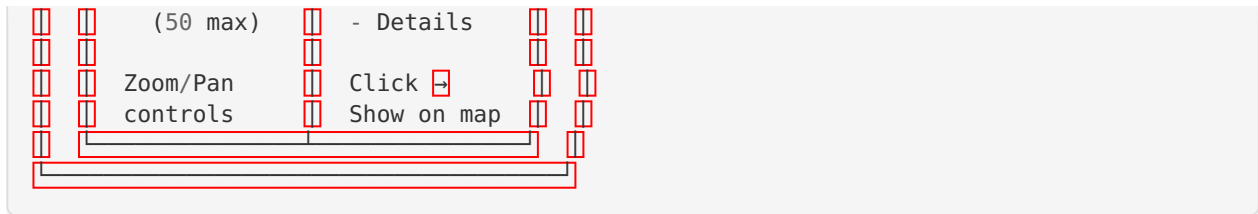
- Cold **start**: ~30s (free tier)

Status:  LIVE AND ACCESSIBLE

Data Flow Diagram

User Search Flow





ML Triage Flow

User Browser

1. Visit `/static/demo.html`

Triage Demo Form

- Complaint: Chest pain
- Age: 45
- Severity: High
- Heart rate: 110 bpm
- BP: 160/100
- Mode: ML (toggle ON)
- Click "Get Recommendation"

2. POST `/api/triage`

```
{
  "chief_complaint": "chest pain",
  "age": 45,
  "severity": "high",
  "heart_rate": 110,
  "systolic_bp": 160,
  "use_ml_model": true
}
```

Backend (main.py)

IF `use_ml_model = True`:

- Load `models/triage_model.pkl`
- Feature engineering:
 - * Encode categorical variables
 - * Normalize vitals
 - * Create interaction features
- Predict with RandomForest:
 - * Triage level (1-5)
 - * Confidence score (%)
 - * Feature importances

ELSE (rule-based):

- Apply `triage_rules.py` logic
- Map symptoms  triage level
- Consider vitals thresholds
- Generate explanation

Return:

- Triage level: "Level 2"
- Confidence: 87%
- Recommendation: "Prioritize time"
- Sort strategy: "ED_wait_time_min"
- Explanation: "High-risk chest pain"

3. JSON Response

Demo Results Display

 Triage Level 2
(High Priority)

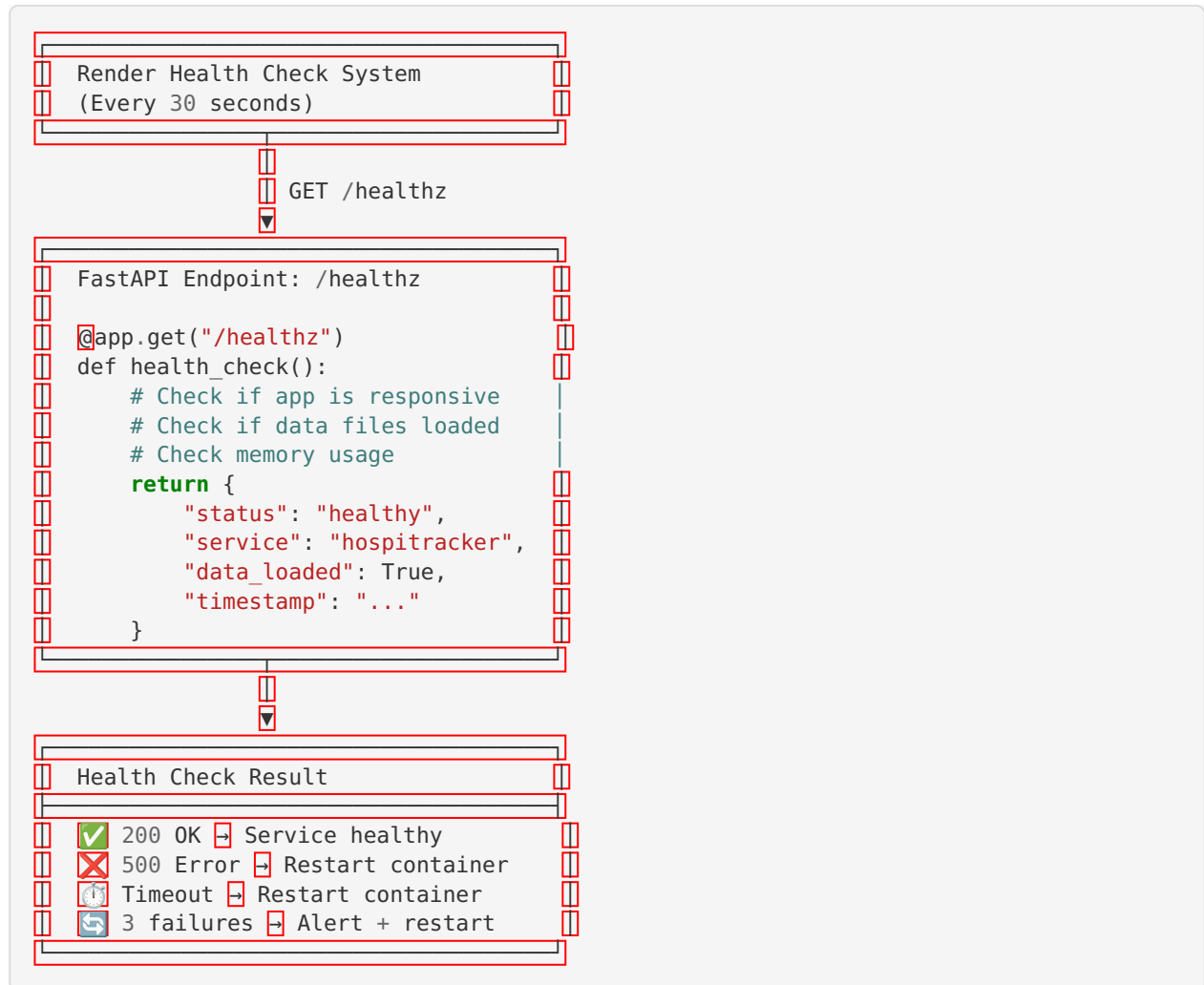
[illegible]

Environment Variables Flow

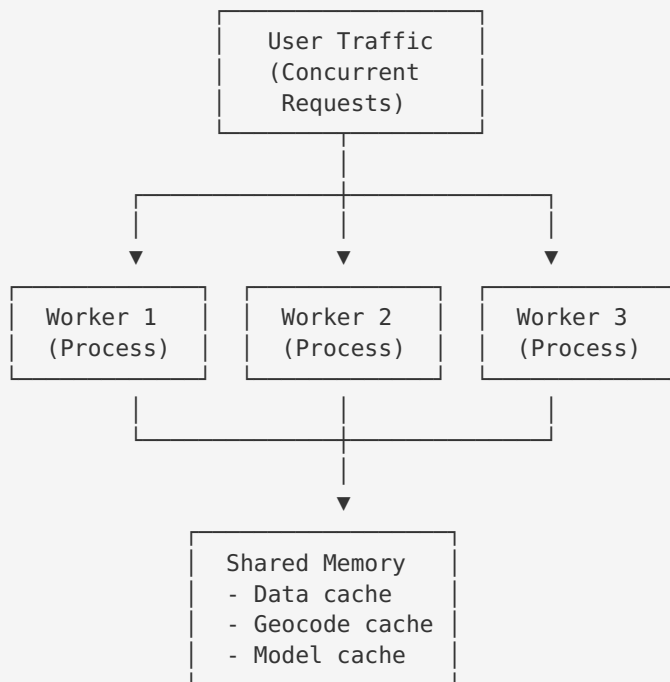
The diagram illustrates the process of injecting environment variables into a Docker container at runtime. It is divided into three main sections:

- Render Dashboard Environment Variables:** A box containing the following variables:
 - PORT=10000
 - PYTHONUNBUFFERED=1
 - HOSPITRACK_DATA_PATH=/app/data
 - GEOCODING_CACHE_SIZE=1000
 - ML_DEMO_ENABLED=true
 - LOG_LEVEL=INFO
- Injected at runtime:** A central box with a downward arrow indicating the flow of data from the dashboard to the container.
- Docker Container:** A box containing the following information:
 - Environment Variables:** A list of variables being injected into the container:
 - os.getenv("PORT") → 10000
 - os.getenv("DATA_PATH") → /app/data
 - os.getenv("GEOCODING_CACHE") → 1000
 - os.getenv("ML_DEMO_ENABLED") → true
 - Usage in code:** A list of actions performed by the application:
 - Unicorn binds to \$PORT
 - Data loader reads \$DATA_PATH
 - Geocoding uses cache size limit
 - ML demo toggles based on flag

Monitoring and Health Check Flow



Scaling and Performance



Performance Characteristics:

- Free tier: 512 MB RAM, shared CPU
- Avg response time: 500ms - 2s
- Max concurrent: ~20-50 requests
- Cold start: ~30s (free tier)
- Throughput: ~100 req/min

Paid tier: (\$7/month)

- 1 GB RAM, dedicated CPU
- Avg response time: 100ms - 500ms
- Max concurrent: ~100-200 requests
- No cold starts (always-on)
- Throughput: ~500 req/min

Deployment Timeline

DEPLOYMENT TIMELINE

- T+0:00 Start deployment
- Push code to GitHub
 - Trigger Render webhook
- T+0:30 Build starts
- Clone repository
 - Docker build initiated
- T+1:00 Dependencies installation
- pip install requirements
 - System package installation
- T+3:00 Copy data **and** models
- COPY data/us_er.parquet
 - COPY models/triage_model.pkl
 - COPY **static**/* (HTML/CSS/JS)
- T+5:00 Build complete
- Push image to registry
 - Deploy to container
- T+6:00 Container startup
- Gunicorn starts
 - Workers spawn
 - Data files **load**
- T+7:00 Health checks begin
- GET /healthz (30s interval)
 - Wait **for** 3 consecutive successes
- T+8:00 🎉 DEPLOYMENT COMPLETE
- Service marked **as** "Live"
 - Public URL accessible
 - Ready to serve traffic
- Total time: ~8 minutes

Rollback Strategy

IF DEPLOYMENT FAILS

Render automatically:

1. Keeps previous deployment running
2. Logs build errors
3. Sends notification
4. Does NOT switch traffic

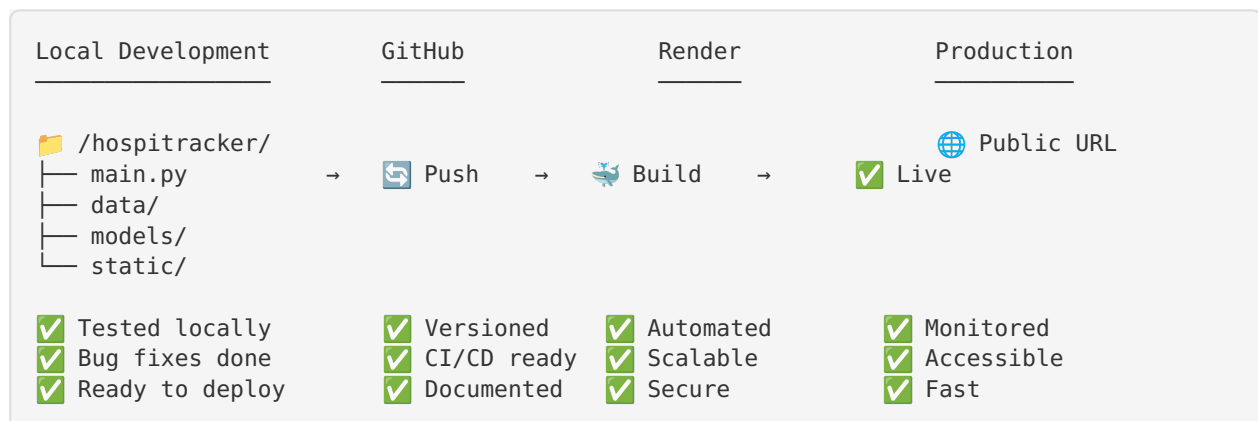
Manual rollback:

- Render Dashboard → Deployments
- Select previous successful build
- Click "Redeploy"

Git rollback:

```
$ git revert HEAD
$ git push origin main
(Triggers new deployment)
```

Summary: From Local to Production



Next Steps: Follow the detailed guide in `ACTUAL_DEPLOYMENT_STEPS.md`